

Course: Advanced Databases

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

Assignment: Idexes 1

1. Objective

The objective of this phase is to propose indexing strategies that improve the performance of the database workload identified and measured in the previous phase.

In Phase 7 (Testing Application), we executed a controlled benchmark using multiple runs with buffer flushing and state restoration to ensure repeatability. The results showed that the total workload execution time averages approximately 10.3 seconds, with relatively low variance across runs, confirming reliable measurements.

However, the analysis clearly identified major performance bottlenecks:

- **Q5 (UPDATE)** is the most expensive operation (~5.8 seconds on average), dominating the workload.
- **Q6 (DELETE)** shows noticeable variability, suggesting inefficient row access.
- **SELECT queries (Q1–Q3)** involve joins, aggregations, and filtering, which result in highly unoptimized access paths.

Based on this, the primary goal of indexing is to:

- reduce data access cost.
- eliminate full table scans where possible.
- improve join and filtering efficiency.
- specifically optimize the most expensive transactions (Q5 and Q6).

2. Proposed Indexing Strategy

To address these performance issues, we propose a combination of different index types: B-tree, Bitmap, and Function-based indexes. These propositions are directly driven by the inefficiencies observed in the EXPLAIN PLAN outputs from Phase 7.

2.1 B-tree Indexes (Primary Optimization)

Index on Activities.runner_id

SQL

```
CREATE INDEX idx_activities_runner_id ON Activities(runner_id);
```

- **Query Plan Justification:** In Phase 7, the EXPLAIN PLAN for Q5 (UPDATE) revealed a massive TABLE ACCESS FULL on the Activities table while trying to match a.runner_id = tg.runner_id. By creating a B-Tree index here, we force the query plan to switch from a costly full scan to a highly efficient INDEX RANGE SCAN, which is expected to reduce Q5's 5.8-second execution time.

Index on Telemetry_Data.activity_id

SQL

```
CREATE INDEX idx_telemetry_activity_id ON Telemetry_Data(activity_id);
```

- **Query Plan Justification:** The execution plans for Q1, Q2, Q3, and Q6 heavily rely on aggregating Telemetry_Data based on activity_id. Currently, the database performs a full table scan on Telemetry_Data before hashing. This index will allow the optimizer to jump directly to the relevant telemetry rows via an INDEX RANGE SCAN.

2.2 Bitmap Index (Low Cardinality Optimization)

Index on Activities.activity_type

SQL

```
CREATE BITMAP INDEX idx_activities_type ON Activities(activity_type);
```

- **Purpose:** Optimize filtering on columns with low cardinality (e.g., activity types such as running).

- **Expected improvement:** Faster filtering and grouping operations in analytical queries, and improved performance for queries using GROUP BY or WHERE on activity type.
- **Note:** Bitmap indexes are efficient for read-heavy operations but may introduce overhead for frequent updates.

2.3 Function-Based Index (Derived Data Optimization)

Index on extracted year from activity_date

SQL

```
CREATE INDEX idx_activity_year ON Activities(EXTRACT(YEAR FROM activity_date));
```

- **Purpose:** Optimize queries filtering by year without recalculating the function at runtime (which currently happens continuously during table scans).
- **Expected improvement:** Faster execution of time-based queries (e.g., Q1, Q2, Q3, and Q6 all filter by the year 2026) by completely removing the computation overhead during the execution phase. It is important to note that these performance gains will be fully observable starting from the first execution after the index has been built and the optimizer updates the execution plan to shift from a full scan to an index-based access path.

3. Mapping Indexes to Workload and Expected Impact

The proposed indexes directly target the operations identified in the workload plan.

- **Q1 (complex SELECT with joins and aggregations):** Improved by idx_activities_runner_id, idx_telemetry_activity_id, and idx_activity_year.
- **Q2 (SELECT with filtering and joins):** Improved by idx_activities_runner_id and idx_activity_year.
- **Q3 (aggregation query):** Improved by idx_activities_type and idx_activity_year.
- **Q4 (INSERT):** Minimal improvement expected (indexes slightly increase insert cost).
- **Q5 (UPDATE):** Improved heavily by idx_activities_runner_id (faster row identification instead of full scan).
- **Q6 (DELETE):** Improved by idx_telemetry_activity_id (faster lookup of related rows).

Overall expected effects:

- Faster join execution.
- Improved row lookup efficiency for UPDATE and DELETE.

- Massive reduction of total workload execution time, especially due to the optimization of Q5.

4. Experimental Plan: Comparing Index Types

To validate the effectiveness of the proposed indexes, we will use the following approach in the next phase:

The Experiment: B-Tree vs. Bitmap on Low Cardinality Data

To prove the specific advantages of different index architectures, we will conduct an A/B comparison experiment.

- **Target:** Queries 1 and 6 both contain the filter `Runners.sex = 'Male'`. The sex column has extremely low cardinality.
- **Methodology:** In Indexes 2 phase, we will run the workload using a standard **B-Tree Index** on `Runners(sex)`. We will log the execution plans and times. Then, we will drop it, replace it with a **Bitmap Index** on `Runners(sex)`, and re-run the workload under identical conditions. The baseline for our comparison is the set of execution times measured without any indexes in Phase 6.
- **Expected Result:** We think that the `EXPLAIN PLAN` for the Bitmap index will show a highly optimized `BITMAP CONVERSION TO ROWID` step, significantly outperforming the B-Tree index in read-heavy queries like Q1, while evaluating the trade-off of potential overhead in data-changing operations.

Indexes 1 Summary

For the overall indexing strategy, the workload will be re-run under the identical conditions established in Phase 7 (flushing `BUFFER_CACHE` and `SHARED_POOL` before each run, and utilizing `ROLLBACK` after each execution).

We will compare the baseline execution times (from Phase 7) against the newly indexed execution times, specifically looking for a noticeable reduction in the execution time of Q5 (the primary bottleneck) and documenting the physical shifts in the `EXPLAIN PLAN` outputs from Full Scans to Index Scans.